VAPTOR Security Assessment Report

Organization: VAPTOR001
Scan ID: SCAN20260320095823
Generated (UTC): 2026-03-20 11:41:52

1) Executive Summary

This assessment summarizes discovered assets, exposed services, and validated findings. MAC addresses are included where available to help correlate devices across scans in DHCP/dynamic IP environments.

Metric	Value
Assets assessed	10
Total vulnerabilities	39
Impact score	30/100
Overall control compliance	0% (0/74)

Severity breakdown

Severity	Count
Critical	0

Severity	Count
High	10
Medium	8
Low	12
Info	9

Assets Scanned

AssetUID	Asset IP	Asset MAC	Open Ports	Vuln Count
VAPTOR001A0001	44.228.249.3	—	—	0
VAPTOR001A0002	44.238.29.244	—	80	39
VAPTOR001A0003	10.0.0.103	3C:0A:F3:3C:C1:A7	—	0
VAPTOR001A0004	10.0.0.113	16:41:0F:EC:FD:25	—	0
VAPTOR001A0005	10.0.0.117	42:27:0A:23:DF:92	—	0
VAPTOR001A0006	10.0.0.120	2A:DD:35:08:D3:72	6700	0
VAPTOR001A0007	10.0.0.254	70:F8:2B:1B:A3:68	53, 80, 443, 1990	0
VAPTOR001A0008	10.0.0.132	B8:27:EB:74:F2:6C	22	0

AssetUID	Asset IP	Asset MAC	Open Ports	Vuln Count
VAPTOR001A0009	10.0.0.106	F4:63:1F:59:71:1C	—	0
VAPTOR001A0010	10.0.0.108	38:53:9C:B8:D1:98	49152, 62078	0

End-of-Life Operating Systems

The following hosts are running operating systems that have reached end-of-life and no longer receive security patches. These represent a critical compliance and security risk and should be upgraded or isolated immediately.

Host IP	Operating System	EOL Since	Recommended Action
10.0.0.132	Linux 4.15 - 5.19	2023-05-31	MEDIUM: Upgrade to 20.04 or 22.04 LTS. If staying on 18.04, enable Ubuntu Pro ESM.
44.238.29.244	Microsoft Windows Server 2012 or Windows Server 2012 R2	2023-10-10	HIGH: Windows Server 2012 is EOL. Migrate to Server 2019/2022. Consider ESU if migration delayed.

Watchdog Answers

- **What is exposed?** 10 assets with 9 open TCP ports; 3 high-risk exposed service(s).
- **What is vulnerable?** 39 validated finding(s) (Critical 0, High 10, Medium 8).
  Top findings: HIGH: SQL Injection - SQLite (Time Based); HIGH: SQL Injection - MsSQL (Time Based); HIGH: SQL Injection

- **What is the evidence?** Evidence snippets present for 39/39 finding(s). See per-asset finding notes for captured headers, fingerprints, and observed behaviors.
- **What is the risk/impact?** Impact score 30/100. Compliance posture: 7% control coverage and 0% vuln-to-control mapping.
- **What should we do next?** Prioritize CRITICAL/HIGH fixes first, then reduce high-risk exposure and re-scan to confirm closure.
  Top actions: Restrict exposure of 10.0.0.132:22 (HIGH). Implement key-based authentication only. Disable password authentication. Use fail2ban. Restrict access by IP. Enable 2FA.; Restrict exposure of 10.0.0.254:80 (HIGH). Redirect all traffic to HTTPS (port 443). Disable HTTP or use only for redirect. Implement HSTS headers.; Restrict exposure of 44.238.29.244:80 (HIGH). Redirect all traffic to HTTPS (port 443). Disable HTTP or use only for redirect. Implement HSTS headers.

2) Technical Summary

Key technical metrics derived from discovery artifacts and assessment outputs:

Metric	Value
Impact score	30/100
Assets discovered	10
Open TCP ports discovered (state=open)	9

Scanner Execution Status

This section records which scanners executed, were skipped, or were unavailable for this run.

Scanner	Status
VAPTORAPI — Vaptor Intelligence	Completed
VAPTORSCOUT — Discovery	Completed
VAPTORRECON — Vulnerability Assessment	Completed
VAPTORWEB — Web Scanner	Completed
VAPTORSQL — SQL Verification	Completed
VAPTORMSF — Exploit Verification	Completed
VAPTORFW — Firewall / IDS Monitoring	Skipped
VAPTORWIFI — WiFi Channel Security Scan	Reserved
VAPTORSET — Social Engineering Assessment	Completed
VAPTORMAST — Mobile Application Security Testing	Completed
VAPTORDAST — Dynamic Application Security Testing	Completed
VAPTORSAST — Static Application Security Testing	Completed

Scanner	Status
VAPTORSHARK — Vaptor Network Shark	Completed
VAPTORAI — Vaptor AI Enhancement	Completed

Validation summary

Stage	Status	Targets	Scanners	Issues	Warnings
Overall	Valid	10	VAPTORMSF, VAPTORSQL, VAPTORWEB	0	0
Vulnerability assessment	Valid	10	VAPTORWEB	0	0
Penetration test	Valid	10	VAPTORMSF, VAPTORSQL	0	0

CMDB Summary

Inventory baseline derived from CMDB tables (MAC-canonical internal devices, IP-canonical external exposure).

Metric	Value
New devices (this scan)	8
New IPs (this scan)	8
IP changes (this scan)	0

Metric	Value
Internal devices (total)	8
Internal devices (seen this scan)	8
Internal devices (unknown vendor)	3
Internal current IPs	8
External IPs (total)	2
External IPs (seen this scan)	2

Top vendors (OUI)

Vendor	Count
Apple, Inc.	1
CLOUD NETWORK TECHNOLOGY SINGAPORE PTE. LTD.	1
DWnet Technologies(Suzhou) Corporation	1
HUAWEI TECHNOLOGIES CO.,LTD	1
Raspberry Pi Foundation	1

Web Recon

Best-effort web content inventory (login hints, emails, paths). Some endpoints may block or return non-standard HTTP.

Metric	Value
Endpoints attempted	2
Endpoints reachable	2
Login hints	2
Emails found	0
Unique paths discovered	17

Endpoint	Status	Login	Emails	Paths	Notes
http://testasp.vulnweb.com/	200	Yes	0	8 (/Default.asp, /Images/logo.gif, /Login.asp)	—
http://testaspnet.vulnweb.com/	200	Yes	0	10 (/Comments.aspx, /ReadNews.aspx, /Signup.aspx)	—

Enrichment

Domain Registration (WHOIS)

IP	Domain	Registrar	Expiration
44.228.249.3	AMAZONAWS.COM	MarkMonitor, Inc.	['2027-01-16T04:59:59+00:00', '2027-01-16T00:00:00+00:00']
44.238.29.244	AMAZONAWS.COM	MarkMonitor, Inc.	['2027-01-16T04:59:59+00:00', '2027-01-16T00:00:00+00:00']

BuiltWith Technology Profile

http://testhtml5.vulnweb.com:

http://testphp.vulnweb.com:

http://testasp.vulnweb.com:

Category	Technologies
Editors	DreamWeaver
Operating Systems	Windows Server
Web Servers	IIS

http://testaspnet.vulnweb.com:

Category	Technologies
Operating Systems	Windows Server
Web Frameworks	Microsoft ASP.NET
Web Servers	IIS

BuiltWith data powered by external API enrichment.

Verification

Verification runs are scoped to discovered open TCP ports and use non-exploit auxiliary probes (service-aware).

Metric	Value
Status	completed
Checks attempted	5
Targets	completed=5, failed=0

Recon (VAPTORSHARK)

Metric	Value
Capture artifacts present	Yes

Metric	Value
Artifact count	6
Total artifact size (bytes)	45410746
Capture interface	eth0
Packets captured	513232
Packets dropped	—
Drop rate	—
PCAP files	1
PCAP total bytes	0
Capture timestamp	2026-03-20T09:50:22.972824Z

Note: Reports intentionally do not include intercepted content (e.g., URLs, credentials) to reduce sensitive-data exposure. Review capture artifacts directly when authorized.

MITM Artifacts (artifacts/mitm)

File	Kind	Bytes
pcap_summary.json	json	723

File	Kind	Bytes
recon_observations.json	json	13658
VAPTORARP20260320113456.log	log	45395105
VAPTORARP20260320113522.log	log	728
VAPTORCAP20260320113522.pcapng	pcap	0
VAPTORPCAP20260320113522.log	log	532

Visited domains (top)

Domain	Count
web.whatsapp.com	6296
dns.msftncsi.com	5162
metadata.google.internal	2833
www.msftconnecttest.com	2518
www.youtube.com	2362
www.linkedin.com	2346
licensing.mp.microsoft.com	2008

Domain	Count
update.eset.com	1926
edge.microsoft.com	1751
ecs.office.com	1748
dcg.microsoft.com	1688
_5222._https.web.whatsapp.com	1566
spot-pa.googleapis.com	1287
pbs.twimg.com	1270
youtube-ui.l.google.com	1046
dynamic.x.com.cdn.cloudflare.net	889
aks-preww-westeurope.access-point.cloudmessaging.edge.microsoft.com	887
r.edtd.eset.com	867
self.events.data.microsoft.com	841
euro02.azure-devices.net	766
v10.events.data.microsoft.com	751

Domain	Count
epns.eset.com	746
gateway.fe2.apple-dns.net	693
e3528.dscg.akamaiedge.net	686
client.wns.windows.com	680

Observed public IPs (top)

IP	Count
40.83.252.19	190713
102.132.99.60	27546
172.217.148.8	12940
105.247.206.196	11858
20.245.38.118	11822
104.18.41.41	10578
91.228.165.43	10564
16.145.39.107	7478

IP	Count
20.44.239.154	6686
40.79.132.142	6399
104.44.44.167	5937
104.44.50.206	5746
199.232.168.158	5394
135.236.137.147	5184
2.20.13.44	4146
20.31.87.92	4076
199.232.214.172	4046
91.228.167.193	4044
91.228.166.14	3026
4.207.247.139	2881
13.107.246.56	2796
72.154.7.106	2270

IP	Count
72.145.35.116	2256
41.21.238.145	2034
20.192.44.82	1991

Hostname enrichment (reverse DNS)

IP	Hostname	Count
102.132.99.60	whatsapp-cdn-shv-01-cpt1.fbcdn.net	27546
172.217.148.8	jnb07s08-in-f8.1e100.net	12940
105.247.206.196	vc-vb-105-247-206-196.ens.vodacom.co.za	11858
91.228.165.43	h1-c03-s.eset.com	10564
16.145.39.107	ec2-16-145-39-107.us-west-2.compute.amazonaws.com	7478
104.44.44.167	ae77-0.ier02.cpt30.ntwk.msn.net	5937
104.44.50.206	ae25-0.icr02.cpt20.ntwk.msn.net	5746
2.20.13.44	a2-20-13-44.deploy.static.akamaitechnologies.com	4146
91.228.167.193	91-228-167-193.ptr.eset.com	4044

IP	Hostname	Count
91.228.166.14	um02.eset.com	3026
41.21.238.145	vc-vb-41-21-238-145.ens.vodacom.co.za	2034
102.132.99.3	whatsapp-chatd-msgr-edge-shv-01-cpt1.facebook.com	1748

Potential sensitive transmissions (signals only)

Signal	Count
http_post	1

SAST - Static Application Security Testing

Source path: /home/***/vaptor/be/tests/Vulnerable-App-main/Vulnerable-App-main

Scan timestamp: 2026-03-20T10:45:32.803020

Total findings: 1

Tool Results

Tool	Status	Findings
bearer	completed	1
bandit	completed	0

Tool	Status	Findings
gitleaks	completed	0

VAPTORSAST Code Security Findings (Bearer)

Severity	Finding	CWE	Location
LOW	Leakage of information in logger message	532	.../christophetd/log4shell/vulnerableapp/MainController.java:18

1 Bearer findings. Ref: artifacts/sast/bearer_results.json

Note: SAST results provide static code analysis findings. Review detailed artifacts under artifacts/sast/ for remediation guidance.

DAST - Dynamic Application Security Testing

Targets scanned: 7

Scan timestamp: 2026-03-20T11:06:37.160397

Total findings: 0

Tool Results

Tool	Status	Targets	Findings
VAPTORRECON	completed	7	0
VAPTORDAST	completed	7	0

Note: DAST results provide dynamic vulnerability assessment findings. Review detailed artifacts under artifacts/dast/ for specific CVE/misconfig details.

VAPTORMAST - Mobile Application Security Testing

App file: /home/***/vaptor/be/tests/pivaa.apk

Package: com.htbridge.pivaa

Version: 1.0 **Min SDK:** 19 **Target SDK:** 26

Scan timestamp: 2026-03-20T11:01:08.462338

Findings: 4 (Critical: 1, High: 1, Medium: 2, Low: 0)

Security Flags

- **debuggable:** True
- **allowBackup:** True

Dangerous Permissions (8)

- android.permission.READ_CONTACTS
- android.permission.ACCESS_COARSE_LOCATION
- android.permission.READ_EXTERNAL_STORAGE

- android.permission.CAMERA
- android.permission.GET_ACCOUNTS
- android.permission.RECORD_AUDIO
- android.permission.ACCESS_FINE_LOCATION
- android.permission.WRITE_EXTERNAL_STORAGE

MAST Findings

Severity	Category	Finding	Detail
HIGH	permissions	Dangerous permissions requested (8)	android.permission.READ_CONTACTS, android.permission.ACCESS_COARSE_LOCATION, android.permission.READ_EXTERNAL_STORAGE, a
CRITICAL	manifest	Application is marked debuggable (android:debuggable=true)	ADB shell access to app private data is possible without root
MEDIUM	manifest	ADB backup allowed (android:allowBackup=true)	ADB backup can extract app data without root access
MEDIUM	certificate	Self-signed certificate	Self-signed certificate

Note: MAST results are from androguard static analysis. Review full results in artifacts/app/appsec_summary.json and artifacts/app/static_analysis.json

VAPTORSET - Social Engineering Assessment

Authorization: ✓ Acknowledged **Actor:** Vaptor **Timestamp:** 2026-03-20T11:40:20.669067+00:00

Credential Captures

Timestamp	Template / Campaign	Source IP / MAC	Offenses	Username
2026-03-20T10:59:11.961342+00:00	form_post	10.0.0.103 / 3C:0A:F3:3C:C1:A7	4	***EMAIL***
2026-03-20T11:36:23.627341+00:00	twitter	10.0.0.103 / 3C:0A:F3:3C:C1:A7	4	***EMAIL***
2026-03-20T11:37:04.509166+00:00	twitter	10.0.0.132 / B8:27:EB:74:F2:6C	3	***EMAIL***
2026-03-20T11:38:47.437696+00:00	office365	10.0.0.132 / B8:27:EB:74:F2:6C	3	***EMAIL***
2026-03-20T11:39:09.697850+00:00	office365	10.0.0.103 / 3C:0A:F3:3C:C1:A7	4	***EMAIL***
2026-03-20T11:40:54.390146+00:00	facebook	10.0.0.132 / B8:27:EB:74:F2:6C	3	***EMAIL***

Timestamp	Template / Campaign	Source IP / MAC	Offenses	Username
2026-03-20T11:41:36.682873+00:00	facebook	10.0.0.103 / 3C:0A:F3:3C:C1:A7	4	***EMAIL***

Errors: Port 8080 is already in use — campaign aborted to avoid conflict with VAPTORWEB or other listeners.

Note: Review full results in artifacts/vaptorset/set_summary.json

3) Technical Details (Per Asset)

Asset: VAPTOR001A0001

Asset fingerprint

Field	Value
MAC Address	N/A
IP	44.228.249.3
Hostname	ec2-44-228-249-3.us-west-2.compute.amazonaws.com
OS fingerprint	N/A

Open ports

None recorded.

Watchdog mini-recap

- **Exposed:** 0 open port(s), 0 high-risk service(s).
- **Vulnerable:** 0 finding(s) (Critical 0, High 0, Medium 0).
- **Evidence:** 0/0 finding(s) include evidence snippets.
- **Remediation:** 0/0 finding(s) include remediation guidance.

Vulnerabilities

CVE/CVSS are shown when available; CVSS may be estimated from severity when not provided by the source.

None found.

Asset: VAPTOR001A0002

Asset fingerprint

Field	Value
MAC Address	N/A
IP	44.238.29.244

Field	Value
Hostname	ec2-44-238-29-244.us-west-2.compute.amazonaws.com
OS fingerprint	Microsoft Windows Server 2012 or Windows Server 2012 R2

Open ports

Proto	Port	Service	Risk
tcp	80	http	HIGH

High-risk port notes (VaptorAI / Intel DB)

Port exposure notes (VaptorAI): tcp/80 http [HIGH]

Narrative

```
# Port Exposure: tcp/80

## Summary
An Internet-accessible service was detected on tcp/80. Exposed management and remote-
access ports are frequently targeted for credential attacks and exploitation.

## Why this matters (Risk)
- Increases attack surface and brute-force exposure
- May permit remote code execution if vulnerable and unpatched
```

```
- Often enables lateral movement once credentials are obtained

## Recommended actions (Remediation)
- Restrict access (VPN/jump host/allowlist)
- Enforce MFA and strong authentication policies
- Patch the exposed service and disable legacy/insecure options

## Evidence
- Observed open: tcp/80
```

Port exposure notes: tcp/80 http [HIGH]

Why this matters Unencrypted web traffic; web application vulnerabilities, MitM attacks

Common attacksSQL Injection, XSS, CSRF, Directory Traversal, Man-in-the-Middle, Session Hijacking

Notable CVEsCVE-2021-44228, CVE-2017-5638, CVE-2014-0160

Recommended action Redirect all traffic to HTTPS (port 443). Disable HTTP or use only for redirect. Implement HSTS headers.

Watchdog mini-recap

- **Exposed:** 1 open port(s), 1 high-risk service(s).
  Top: tcp/80 http [HIGH]
- **Vulnerable:** 39 finding(s) (Critical 0, High 10, Medium 8).

- **Evidence:** 39/39 finding(s) include evidence snippets.
- **Remediation:** 37/39 finding(s) include remediation guidance.
  Suggested: Do not trust client side input, even if there is client side validation in place.In general, type check all data on the server side.If the application uses JDBC, use PreparedSta...; Do not trust client side input, even if there is client side validation in place.In general, type check all data on the server side.If the application uses JDBC, use PreparedSta...

Vulnerabilities (39)

CVE/CVSS are shown when available; CVSS may be estimated from severity when not provided by the source.

Severity	Title	CVE	CVSS
HIGH	SQL Injection - SQLite (Time Based)	—	8.0
HIGH	SQL Injection - MsSQL (Time Based)	—	8.0
HIGH	SQL Injection	—	8.0
HIGH	Cross Site Scripting (Reflected)	—	8.0
HIGH	Cross Site Scripting (Persistent)	—	8.0
HIGH	SQL Injection - MsSQL (Time Based)	—	8.0
HIGH	SQL Injection	—	8.0

Severity	Title	CVE	CVSS
HIGH	Path Traversal	—	8.0
HIGH	External Redirect	—	8.0
HIGH	Cross Site Scripting (Reflected)	—	8.0
MEDIUM	Missing Anti-clickjacking Header	—	5.0
MEDIUM	HTTP Only Site	—	5.0
MEDIUM	Content Security Policy (CSP) Header Not Set	—	5.0
MEDIUM	Absence of Anti-CSRF Tokens	—	5.0
MEDIUM	Missing Anti-clickjacking Header	—	5.0
MEDIUM	HTTP Only Site	—	5.0
MEDIUM	Content Security Policy (CSP) Header Not Set	—	5.0
MEDIUM	Absence of Anti-CSRF Tokens	—	5.0
LOW	X-Content-Type-Options Header Missing	—	2.5
LOW	X-AspNet-Version Response Header	—	2.5
LOW	Server Leaks Version Information via "Server" HTTP Response Header Field	—	2.5

Severity	Title	CVE	CVSS
LOW	Server Leaks Information via "X-Powered-By" HTTP Response Header Field(s)	—	2.5
LOW	Cookie without SameSite Attribute	—	2.5
LOW	X-Content-Type-Options Header Missing	—	2.5
LOW	Server Leaks Version Information via "Server" HTTP Response Header Field	—	2.5
LOW	Server Leaks Information via "X-Powered-By" HTTP Response Header Field(s)	—	2.5
LOW	Information Disclosure - Debug Error Messages	—	2.5
LOW	Cookie without SameSite Attribute	—	2.5
LOW	Cookie No HttpOnly Flag	—	2.5
LOW	Application Error Disclosure	—	2.5
INFO	User Controllable HTML Element Attribute (Potential XSS)	—	0.0
INFO	User Agent Fuzzer	—	0.0
INFO	Session Management Response Identified	—	0.0
INFO	GET for POST	—	0.0
INFO	Charset Mismatch (Header Versus Meta Content-Type Charset)	—	0.0

Severity	Title	CVE	CVSS
INFO	Authentication Request Identified	—	0.0
INFO	User Agent Fuzzer	—	0.0
INFO	Session Management Response Identified	—	0.0
INFO	Information Disclosure - Suspicious Comments	—	0.0

Finding notes: SQL Injection - SQLite (Time Based)

Description

```
SQL injection may be possible.
```

Evidence

```
Target: http://testaspnet.vulnweb.com/Comments.aspx?id=2 | Finding: SQL Injection -
SQLite (Time Based) | Reported Severity: high
```

Remediation

```
Do not trust client side input, even if there is client side validation in place.In
general, type check all data on the server side.If the application uses JDBC, use
PreparedStatement or CallableStatement, with parameters passed by '?'If the application
uses ASP, use ADO Command Objects with strong type checking and parameterized queries.If
database Stored Procedures can be used, use them.Do *not* concatenate strings into
```

```
queries in the stored procedure, or use 'exec', 'exec immediate', or equivalent
functionality!Do not create dynamic SQL queries using simple string concatenation.Escape
all data received from the client.Apply an 'allow list' of allowed characters, or a 'deny
list' of disallowed characters in user input.Apply the principle of least privilege by
using the least privileged database user possible.In particular, avoid using the 'sa' or
'db-owner' database users. This does not eliminate SQL injection, but minimizes its
impact.Grant the minimum database access that is necessary for the application.
```

Remediation (detailed)

Effort: 4-8 hours per affected query
Business impact: CRITICAL - Can lead to complete data breach, data manipulation, or system compromise

1. Use parameterized queries (prepared statements) for all database interactions
2. Implement input validation with whitelist approach
3. Apply principle of least privilege for database accounts
4. Enable database query logging and monitoring
5. Use ORM frameworks with proper configuration

Verification:

- Test with common SQL injection payloads (e.g., ' OR '1'='1)
- Use automated scanners (VAPTORSQL, Burp Suite) to verify fix
- Review code for dynamic query construction
- Implement automated security testing in CI/CD pipeline

References:

- https://cheatsheetseries.owasp.org/cheatsheets/SQL_Injection_Prevention_Cheat_Sheet.html

- https://owasp.org/www-community/attacks/SQL_Injection
- https://cwe.mitre.org/data/definitions/89.html

Finding notes: SQL Injection - MsSQL (Time Based)

Description

```
SQL injection may be possible.
```

Evidence

```
Target: http://testaspnet.vulnweb.com/ReadNews.aspx?NewsAd=ads/def.html&id=0 | Finding:
SQL Injection - MsSQL (Time Based) | Reported Severity: high
```

Remediation

```
Do not trust client side input, even if there is client side validation in place.In
general, type check all data on the server side.If the application uses JDBC, use
PreparedStatement or CallableStatement, with parameters passed by '?'If the application
uses ASP, use ADO Command Objects with strong type checking and parameterized queries.If
database Stored Procedures can be used, use them.Do *not* concatenate strings into
queries in the stored procedure, or use 'exec', 'exec immediate', or equivalent
functionality!Do not create dynamic SQL queries using simple string concatenation.Escape
all data received from the client.Apply an 'allow list' of allowed characters, or a 'deny
list' of disallowed characters in user input.Apply the principle of least privilege by
using the least privileged database user possible.In particular, avoid using the 'sa' or
```

```
'db-owner' database users. This does not eliminate SQL injection, but minimizes its
impact.Grant the minimum database access that is necessary for the application.
```

Remediation (detailed)

Effort: 4-8 hours per affected query
Business impact: CRITICAL - Can lead to complete data breach, data manipulation, or system compromise

1. Use parameterized queries (prepared statements) for all database interactions
2. Implement input validation with whitelist approach
3. Apply principle of least privilege for database accounts
4. Enable database query logging and monitoring
5. Use ORM frameworks with proper configuration

Verification:

- Test with common SQL injection payloads (e.g., ' OR '1'='1)
- Use automated scanners (VAPTORSQL, Burp Suite) to verify fix
- Review code for dynamic query construction
- Implement automated security testing in CI/CD pipeline

References:

- https://cheatsheetseries.owasp.org/cheatsheets/SQL_Injection_Prevention_Cheat_Sheet.html
- https://owasp.org/www-community/attacks/SQL_Injection
- https://cwe.mitre.org/data/definitions/89.html

Finding notes: SQL Injection

Description

```
SQL injection may be possible.
```

Evidence

```
Target: http://testaspnet.vulnweb.com/Comments.aspx?id=4-2 | Finding: SQL Injection |
Reported Severity: high
```

Remediation

```
Do not trust client side input, even if there is client side validation in place.In
general, type check all data on the server side.If the application uses JDBC, use
PreparedStatement or CallableStatement, with parameters passed by '?'If the application
uses ASP, use ADO Command Objects with strong type checking and parameterized queries.If
database Stored Procedures can be used, use them.Do *not* concatenate strings into
queries in the stored procedure, or use 'exec', 'exec immediate', or equivalent
functionality!Do not create dynamic SQL queries using simple string concatenation.Escape
all data received from the client.Apply an 'allow list' of allowed characters, or a 'deny
list' of disallowed characters in user input.Apply the principle of least privilege by
using the least privileged database user possible.In particular, avoid using the 'sa' or
'db-owner' database users. This does not eliminate SQL injection, but minimizes its
impact.Grant the minimum database access that is necessary for the application.
```

Remediation (detailed)

Effort: 4-8 hours per affected query

Business impact: CRITICAL - Can lead to complete data breach, data manipulation, or system compromise

1. Use parameterized queries (prepared statements) for all database interactions
2. Implement input validation with whitelist approach
3. Apply principle of least privilege for database accounts
4. Enable database query logging and monitoring
5. Use ORM frameworks with proper configuration

Verification:

- Test with common SQL injection payloads (e.g., ' OR '1'='1)
- Use automated scanners (VAPTORSQL, Burp Suite) to verify fix
- Review code for dynamic query construction
- Implement automated security testing in CI/CD pipeline

References:

- https://cheatsheetseries.owasp.org/cheatsheets/SQL_Injection_Prevention_Cheat_Sheet.html
- https://owasp.org/www-community/attacks/SQL_Injection
- https://cwe.mitre.org/data/definitions/89.html

Finding notes: Cross Site Scripting (Reflected)

Description

```
Cross-site Scripting (XSS) is an attack technique that involves echoing attacker-supplied
code into a user's browser instance. A browser instance can be a standard web browser
client, or a browser object embedded in a software product such as the browser within
```

WinAmp, an RSS reader, or an email client. The code itself is usually written in HTML/JavaScript, but may also extend to VBScript, ActiveX, Java, Flash, or any other browser-supported technology.When an attacker gets a user's browser to execute his/her code, the code will run within the security context (or zone) of the hosting web site. With this level of privilege, the code has the ability to read, modify and transmit any sensitive data accessible by the browser. A Cross-site Scripted user could have his/her account hijacked (cookie theft), their browser redirected to another location, or possibly shown fraudulent content delivered by the web site they are visiting. Cross-site Scripting attacks essentially compromise the trust relationship between a user and the web site. Applications utilizing browser object instances which load content from the file system may execute code under the local machine zone allowing for system compromise.There are three types of Cross-site Scripting attacks: non-persistent, persistent and DOM-based.Non-persistent attacks and DOM-based attacks require a user to either visit a specially crafted link laced with malicious code, or visit a malicious web page containing a web form, which when posted to the vulnerable site, will mount the attack. Using a malicious form will oftentimes take place when the vulnerable resource only accepts HTTP POST requests. In such a case, the form can be submitted automatically, without the victim's knowledge (e.g. by using JavaScript). Upon clicking on the malicious link or submitting the malicious form, the XSS payload will get echoed back and will get interpreted by the user's browser and execute. Another technique to send almost arbitrary requests (GET and POST) is by using an embedded client, such as Adobe Flash.Persistent attacks occur when the malicious code is submitted to a web site where it's stored for a period of time. Examples of a...

Evidence

```
Target: http://testaspnet.vulnweb.com/ReadNews.aspx?
NewsAd=javascript%3Aalert%281%29%3B&id=0 | Finding: Cross Site Scripting (Reflected) |
Reported Severity: high
```

Remediation

```
Phase: Architecture and DesignUse a vetted library or framework that does not allow this
weakness to occur or provides constructs that make this weakness easier to avoid.Examples
of libraries and frameworks that make it easier to generate properly encoded output
include Microsoft's Anti-XSS library, the OWASP ESAPI Encoding module, and Apache
Wicket.Phases: Implementation; Architecture and DesignUnderstand the context in which
your data will be used and the encoding that will be expected. This is especially
important when transmitting data between different components, or when generating outputs
that can contain multiple encodings at the same time, such as web pages or multi-part
mail messages. Study all expected communication protocols and data representations to
determine the required encoding strategies.For any data that will be output to another
web page, especially any data that was received from external inputs, use the appropriate
encoding on all non-alphanumeric characters.Consult the XSS Prevention Cheat Sheet for
more details on the types of encoding and escaping that are needed.Phase: Architecture
and DesignFor any security checks that are performed on the client side, ensure that
these checks are duplicated on the server side, in order to avoid CWE-602. Attackers can
bypass the client-side checks by modifying values after the checks have been performed,
or by changing the client to remove the client-side checks entirely. Then, these modified
```

```
values would be submitted to the server.If available, use structured mechanisms that
automatically enforce the separati...
```

Remediation (detailed)

Effort: Variable
Business impact: Depends on vulnerability context

1. Phase: Architecture and DesignUse a vetted library or framework that does not allow this weakness to occur or provides constructs that make this weakness easier to avoid.Examples of libraries and frameworks that make it easier to generate properly encoded output include Microsoft's Anti-XSS library, the OWASP ESAPI Encoding module, and Apache Wicket.Phases: Implementation; Architecture and DesignUnderstand the con...

References:

- https://owasp.org/www-community/attacks/xss/
- https://cwe.mitre.org/data/definitions/79.html

Finding notes: Cross Site Scripting (Persistent)

Description

```
Cross-site Scripting (XSS) is an attack technique that involves echoing attacker-supplied
code into a user's browser instance. A browser instance can be a standard web browser
client, or a browser object embedded in a software product such as the browser within
WinAmp, an RSS reader, or an email client. The code itself is usually written in HTML/
JavaScript, but may also extend to VBScript, ActiveX, Java, Flash, or any other browser-
```

supported technology.When an attacker gets a user's browser to execute his/her code, the code will run within the security context (or zone) of the hosting web site. With this level of privilege, the code has the ability to read, modify and transmit any sensitive data accessible by the browser. A Cross-site Scripted user could have his/her account hijacked (cookie theft), their browser redirected to another location, or possibly shown fraudulent content delivered by the web site they are visiting. Cross-site Scripting attacks essentially compromise the trust relationship between a user and the web site. Applications utilizing browser object instances which load content from the file system may execute code under the local machine zone allowing for system compromise.There are three types of Cross-site Scripting attacks: non-persistent, persistent and DOM-based.Non-persistent attacks and DOM-based attacks require a user to either visit a specially crafted link laced with malicious code, or visit a malicious web page containing a web form, which when posted to the vulnerable site, will mount the attack. Using a malicious form will oftentimes take place when the vulnerable resource only accepts HTTP POST requests. In such a case, the form can be submitted automatically, without the victim's knowledge (e.g. by using JavaScript). Upon clicking on the malicious link or submitting the malicious form, the XSS payload will get echoed back and will get interpreted by the user's browser and execute. Another technique to send almost arbitrary requests (GET and POST) is by using an embedded client, such as Adobe Flash.Persistent attacks occur when the malicious code is submitted to a web site where it's stored for a period of time. Examples of a...

Evidence

```
Target: http://testaspnet.vulnweb.com/Comments.aspx?id=2 | Finding: Cross Site Scripting
(Persistent) | Reported Severity: high
```

Remediation

```
Phase: Architecture and DesignUse a vetted library or framework that does not allow this
weakness to occur or provides constructs that make this weakness easier to avoid.Examples
of libraries and frameworks that make it easier to generate properly encoded output
include Microsoft's Anti-XSS library, the OWASP ESAPI Encoding module, and Apache
Wicket.Phases: Implementation; Architecture and DesignUnderstand the context in which
your data will be used and the encoding that will be expected. This is especially
important when transmitting data between different components, or when generating outputs
that can contain multiple encodings at the same time, such as web pages or multi-part
mail messages. Study all expected communication protocols and data representations to
determine the required encoding strategies.For any data that will be output to another
web page, especially any data that was received from external inputs, use the appropriate
encoding on all non-alphanumeric characters.Consult the XSS Prevention Cheat Sheet for
more details on the types of encoding and escaping that are needed.Phase: Architecture
and DesignFor any security checks that are performed on the client side, ensure that
these checks are duplicated on the server side, in order to avoid CWE-602. Attackers can
bypass the client-side checks by modifying values after the checks have been performed,
or by changing the client to remove the client-side checks entirely. Then, these modified
values would be submitted to the server.If available, use structured mechanisms that
automatically enforce the separati...
```

Remediation (detailed)

Effort: Variable
Business impact: Depends on vulnerability context

1. Phase: Architecture and DesignUse a vetted library or framework that does not allow this weakness to occur or provides constructs that make this weakness easier to avoid.Examples of libraries and frameworks that make it easier to generate properly encoded output include Microsoft's Anti-XSS library, the OWASP ESAPI Encoding module, and Apache Wicket.Phases: Implementation; Architecture and DesignUnderstand the con...

References:

- https://owasp.org/www-community/attacks/xss/
- https://cwe.mitre.org/data/definitions/79.html

Finding notes: SQL Injection - MsSQL (Time Based)

Description

```
SQL injection may be possible.
```

Evidence

```
Target: http://testasp.vulnweb.com/showforum.asp?id=0 | Finding: SQL Injection - MsSQL
(Time Based) | Reported Severity: high
```

Remediation

```
Do not trust client side input, even if there is client side validation in place.In
general, type check all data on the server side.If the application uses JDBC, use
PreparedStatement or CallableStatement, with parameters passed by '?'If the application
uses ASP, use ADO Command Objects with strong type checking and parameterized queries.If
database Stored Procedures can be used, use them.Do *not* concatenate strings into
queries in the stored procedure, or use 'exec', 'exec immediate', or equivalent
functionality!Do not create dynamic SQL queries using simple string concatenation.Escape
all data received from the client.Apply an 'allow list' of allowed characters, or a 'deny
list' of disallowed characters in user input.Apply the principle of least privilege by
using the least privileged database user possible.In particular, avoid using the 'sa' or
'db-owner' database users. This does not eliminate SQL injection, but minimizes its
impact.Grant the minimum database access that is necessary for the application.
```

Remediation (detailed)

Effort: 4-8 hours per affected query
Business impact: CRITICAL - Can lead to complete data breach, data manipulation, or system compromise

1. Use parameterized queries (prepared statements) for all database interactions
2. Implement input validation with whitelist approach
3. Apply principle of least privilege for database accounts
4. Enable database query logging and monitoring
5. Use ORM frameworks with proper configuration

Verification:

- Test with common SQL injection payloads (e.g., ' OR '1'='1)

- Use automated scanners (VAPTORSQL, Burp Suite) to verify fix
- Review code for dynamic query construction
- Implement automated security testing in CI/CD pipeline

References:

- https://cheatsheetseries.owasp.org/cheatsheets/SQL_Injection_Prevention_Cheat_Sheet.html
- https://owasp.org/www-community/attacks/SQL_Injection
- https://cwe.mitre.org/data/definitions/89.html

Finding notes: SQL Injection

Description

```
SQL injection may be possible.
```

Evidence

```
Target: http://testasp.vulnweb.com/Search.asp?tfSearch=VAPTORWEB%27 | Finding: SQL
Injection | Reported Severity: high
```

Remediation

```
Do not trust client side input, even if there is client side validation in place.In
general, type check all data on the server side.If the application uses JDBC, use
PreparedStatement or CallableStatement, with parameters passed by '?'If the application
uses ASP, use ADO Command Objects with strong type checking and parameterized queries.If
database Stored Procedures can be used, use them.Do *not* concatenate strings into
```

```
queries in the stored procedure, or use 'exec', 'exec immediate', or equivalent
functionality!Do not create dynamic SQL queries using simple string concatenation.Escape
all data received from the client.Apply an 'allow list' of allowed characters, or a 'deny
list' of disallowed characters in user input.Apply the principle of least privilege by
using the least privileged database user possible.In particular, avoid using the 'sa' or
'db-owner' database users. This does not eliminate SQL injection, but minimizes its
impact.Grant the minimum database access that is necessary for the application.
```

Remediation (detailed)

Effort: 4-8 hours per affected query
Business impact: CRITICAL - Can lead to complete data breach, data manipulation, or system compromise

1. Use parameterized queries (prepared statements) for all database interactions
2. Implement input validation with whitelist approach
3. Apply principle of least privilege for database accounts
4. Enable database query logging and monitoring
5. Use ORM frameworks with proper configuration

Verification:

- Test with common SQL injection payloads (e.g., ' OR '1'='1)
- Use automated scanners (VAPTORSQL, Burp Suite) to verify fix
- Review code for dynamic query construction
- Implement automated security testing in CI/CD pipeline

References:

- https://cheatsheetseries.owasp.org/cheatsheets/SQL_Injection_Prevention_Cheat_Sheet.html

- https://owasp.org/www-community/attacks/SQL_Injection
- https://cwe.mitre.org/data/definitions/89.html

Finding notes: Path Traversal

Description

```
The Path Traversal attack technique allows an attacker access to files, directories, and
commands that potentially reside outside the web document root directory. An attacker may
manipulate a URL in such a way that the web site will execute or reveal the contents of
arbitrary files anywhere on the web server. Any device that exposes an HTTP-based
interface is potentially vulnerable to Path Traversal.Most web sites restrict user access
to a specific portion of the file-system, typically called the "web document root" or
"CGI root" directory. These directories contain the files intended for user access and
the executable necessary to drive web application functionality. To access files or
execute commands anywhere on the file-system, Path Traversal attacks will utilize the
ability of special-characters sequences.The most basic Path Traversal attack uses the
"../" special-character sequence to alter the resource location requested in the URL.
Although most popular web servers will prevent this technique from escaping the web
document root, alternate encodings of the "../" sequence may help bypass the security
filters. These method variations include valid and invalid Unicode-encoding ("..%u2216"
or "..%c0%af") of the forward slash character, backslash characters ("..\") on Windows-
based servers, URL encoded characters "%2e%2e%2f"), and double URL encoding ("..%255c")
of the backslash character.Even if the web server properly restricts Path Traversal
attempts in the URL path, a web application itself may still be vulnerable due to
```

```
improper handling of user-supplied input. This is a common problem of web applications
that use template mechanisms or load static text from files. In variations of the attack,
the original URL parameter value is substituted with the file name of one of the web
application's dynamic scripts. Consequently, the results can reveal source code because
the file is interpreted as text instead of an executable script. These techniques often
employ additional special characters such as the dot (".") to reveal the listing of the
current working directory, or "%00" NULL characters in order to bypass rudimentary file
extension checks.
```

Evidence

```
Target: http://testasp.vulnweb.com/Register.asp?RetURL=%2FDefault%2Easp%3F | Finding:
Path Traversal | Reported Severity: high
```

Remediation

```
Assume all input is malicious. Use an "accept known good" input validation strategy,
i.e., use an allow list of acceptable inputs that strictly conform to specifications.
Reject any input that does not strictly conform to specifications, or transform it into
something that does. Do not rely exclusively on looking for malicious or malformed inputs
(i.e., do not rely on a deny list). However, deny lists can be useful for detecting
potential attacks or determining which inputs are so malformed that they should be
rejected outright.When performing input validation, consider all potentially relevant
properties, including length, type of input, the full range of acceptable values, missing
or extra inputs, syntax, consistency across related fields, and conformance to business
rules. As an example of business rule logic, "boat" may be syntactically valid because it
```

```
only contains alphanumeric characters, but it is not valid if you are expecting colors
such as "red" or "blue."For filenames, use stringent allow lists that limit the character
set to be used. If feasible, only allow a single "." character in the filename to avoid
weaknesses, and exclude directory separators such as "/". Use an allow list of allowable
file extensions.Warning: if you attempt to cleanse your data, then do so that the end
result is not in the form that can be dangerous. A sanitizing mechanism can remove
characters such as '.' and ';' which may be required for some exploits. An attacker can
try to fool the sanitizing mechanism into "cleaning" data into a dangerous form. Suppose
the attacker injects a '.' ...
```

Remediation (detailed)

Effort: 2-4 hours
Business impact: HIGH - Prevents unauthorized file access and information disclosure

1. Validate and sanitize all file path inputs
2. Use whitelist of allowed files/directories
3. Avoid passing user input directly to file system APIs
4. Use path canonicalization to resolve relative paths
5. Implement proper access controls on file system

Verification:

- Test with ../../../etc/passwd
- Test with URL-encoded traversal: ..%2F..%2F..%2Fetc%2Fpasswd
- Test with double encoding
- Verify whitelist is enforced

References:

- https://owasp.org/www-community/attacks/Path_Traversal
- https://cwe.mitre.org/data/definitions/22.html

Finding notes: External Redirect

Description

```
URL redirectors represent common functionality employed by web sites to forward an
incoming request to an alternate resource. This can be done for a variety of reasons and
is often done to allow resources to be moved within the directory structure and to avoid
breaking functionality for users that request the resource at its previous location. URL
redirectors may also be used to implement load balancing, leveraging abbreviated URLs or
recording outgoing links. It is this last implementation which is often used in phishing
attacks as described in the example below. URL redirectors do not necessarily represent a
direct security vulnerability but can be abused by attackers trying to social engineer
victims into believing that they are navigating to a site other than the true
destination.
```

Evidence

```
Target: http://testasp.vulnweb.com/Logout.asp?
RetURL=https%3A%2F%2F9217463592712781262.owasp.org | Finding: External Redirect |
Reported Severity: high
```

Remediation

```
Assume all input is malicious. Use an "accept known good" input validation strategy,
i.e., use an allow list of acceptable inputs that strictly conform to specifications.
Reject any input that does not strictly conform to specifications, or transform it into
something that does. Do not rely exclusively on looking for malicious or malformed inputs
(i.e., do not rely on a deny list). However, deny lists can be useful for detecting
potential attacks or determining which inputs are so malformed that they should be
rejected outright.When performing input validation, consider all potentially relevant
properties, including length, type of input, the full range of acceptable values, missing
or extra inputs, syntax, consistency across related fields, and conformance to business
rules. As an example of business rule logic, "boat" may be syntactically valid because it
only contains alphanumeric characters, but it is not valid if you are expecting colors
such as "red" or "blue."Use an allow list of approved URLs or domains to be used for
redirection.Use an intermediate disclaimer page that provides the user with a clear
warning that they are leaving your site. Implement a long timeout before the redirect
occurs, or force the user to click on the link. Be careful to avoid XSS problems when
generating the disclaimer page.When the set of acceptable objects, such as filenames or
URLs, is limited or known, create a mapping from a set of fixed input values (such as
numeric IDs) to the actual filenames or URLs, and reject all other inputs.For example, ID
1 could map to "/login.asp" and ID ...
```

Remediation (detailed)

Effort: Variable

Business impact: Depends on vulnerability context

1. Assume all input is malicious. Use an "accept known good" input validation strategy, i.e., use an allow list of acceptable inputs that strictly conform to specifications. Reject any input that does not strictly conform to specifications, or transform it into something that does. Do not rely exclusively on looking for malicious or malformed inputs (i.e., do not rely on a deny list). However, deny lists can be usefu...

References:

- https://cheatsheetseries.owasp.org/cheatsheets/Unvalidated_Redirects_and_Forwards_Cheat_Sheet.html
- https://cwe.mitre.org/data/definitions/601.html

Finding notes: Cross Site Scripting (Reflected)

Description

```
Cross-site Scripting (XSS) is an attack technique that involves echoing attacker-supplied
code into a user's browser instance. A browser instance can be a standard web browser
client, or a browser object embedded in a software product such as the browser within
WinAmp, an RSS reader, or an email client. The code itself is usually written in HTML/
JavaScript, but may also extend to VBScript, ActiveX, Java, Flash, or any other browser-
supported technology.When an attacker gets a user's browser to execute his/her code, the
code will run within the security context (or zone) of the hosting web site. With this
level of privilege, the code has the ability to read, modify and transmit any sensitive
data accessible by the browser. A Cross-site Scripted user could have his/her account
hijacked (cookie theft), their browser redirected to another location, or possibly shown
```

```
fraudulent content delivered by the web site they are visiting. Cross-site Scripting
attacks essentially compromise the trust relationship between a user and the web site.
Applications utilizing browser object instances which load content from the file system
may execute code under the local machine zone allowing for system compromise.There are
three types of Cross-site Scripting attacks: non-persistent, persistent and DOM-
based.Non-persistent attacks and DOM-based attacks require a user to either visit a
specially crafted link laced with malicious code, or visit a malicious web page
containing a web form, which when posted to the vulnerable site, will mount the attack.
Using a malicious form will oftentimes take place when the vulnerable resource only
accepts HTTP POST requests. In such a case, the form can be submitted automatically,
without the victim's knowledge (e.g. by using JavaScript). Upon clicking on the malicious
link or submitting the malicious form, the XSS payload will get echoed back and will get
interpreted by the user's browser and execute. Another technique to send almost arbitrary
requests (GET and POST) is by using an embedded client, such as Adobe Flash.Persistent
attacks occur when the malicious code is submitted to a web site where it's stored for a
period of time. Examples of a...
```

Evidence

```
Target: http://testasp.vulnweb.com/showthread.asp?id=4 | Finding: Cross Site Scripting
(Reflected) | Reported Severity: high
```

Remediation

```
Phase: Architecture and DesignUse a vetted library or framework that does not allow this
weakness to occur or provides constructs that make this weakness easier to avoid.Examples
```

```
of libraries and frameworks that make it easier to generate properly encoded output
include Microsoft's Anti-XSS library, the OWASP ESAPI Encoding module, and Apache
Wicket.Phases: Implementation; Architecture and DesignUnderstand the context in which
your data will be used and the encoding that will be expected. This is especially
important when transmitting data between different components, or when generating outputs
that can contain multiple encodings at the same time, such as web pages or multi-part
mail messages. Study all expected communication protocols and data representations to
determine the required encoding strategies.For any data that will be output to another
web page, especially any data that was received from external inputs, use the appropriate
encoding on all non-alphanumeric characters.Consult the XSS Prevention Cheat Sheet for
more details on the types of encoding and escaping that are needed.Phase: Architecture
and DesignFor any security checks that are performed on the client side, ensure that
these checks are duplicated on the server side, in order to avoid CWE-602. Attackers can
bypass the client-side checks by modifying values after the checks have been performed,
or by changing the client to remove the client-side checks entirely. Then, these modified
values would be submitted to the server.If available, use structured mechanisms that
automatically enforce the separati...
```

Remediation (detailed)

Effort: Variable

Business impact: Depends on vulnerability context

1. Phase: Architecture and DesignUse a vetted library or framework that does not allow this weakness to occur or provides constructs that make this weakness easier to avoid.Examples of libraries and frameworks that make it easier to generate properly encoded output include Microsoft's Anti-XSS

library, the OWASP ESAPI Encoding module, and Apache Wicket.Phases: Implementation; Architecture and DesignUnderstand the con...

References:

- https://owasp.org/www-community/attacks/xss/
- https://cwe.mitre.org/data/definitions/79.html

Finding notes: Missing Anti-clickjacking Header

Description

```
The response does not protect against 'ClickJacking' attacks. It should include either
Content-Security-Policy with 'frame-ancestors' directive or X-Frame-Options.
```

Evidence

```
Target: http://testaspnet.vulnweb.com | Finding: Missing Anti-clickjacking Header |
Reported Severity: medium
```

Remediation

```
Modern Web browsers support the Content-Security-Policy and X-Frame-Options HTTP headers.
Ensure one of them is set on all web pages returned by your site/app.If you expect the
page to be framed only by pages on your server (e.g. it's part of a FRAMESET) then you'll
want to use SAMEORIGIN, otherwise if you never expect the page to be framed, you should
```

```
use DENY. Alternatively consider implementing Content Security Policy's "frame-ancestors"
directive.
```

Remediation (detailed)

Effort: Variable
Business impact: Depends on vulnerability context

1. Modern Web browsers support the Content-Security-Policy and X-Frame-Options HTTP headers. Ensure one of them is set on all web pages returned by your site/app.If you expect the page to be framed only by pages on your server (e.g. it's part of a FRAMESET) then you'll want to use SAMEORIGIN, otherwise if you never expect the page to be framed, you should use DENY. Alternatively consider implementing Content Security...

References:

- https://developer.mozilla.org/en-US/docs/Web/HTTP/Reference/Headers/X-Frame-Options

Finding notes: HTTP Only Site

Description

```
The site is only served under HTTP and not HTTPS.
```

Evidence

```
Target: http://testaspnet.vulnweb.com/Comments.aspx?id=2 | Finding: HTTP Only Site |
Reported Severity: medium
```

Remediation

```
Configure your web or application server to use SSL (https).
```

Remediation (detailed)

Effort: Variable
Business impact: Depends on vulnerability context

1. Configure your web or application server to use SSL (https).

References:

- https://cheatsheetseries.owasp.org/cheatsheets/Transport_Layer_Protection_Cheat_Sheet.html
- https://letsencrypt.org/

Asset: VAPTOR001A0003

Asset fingerprint

Field	Value
MAC Address	3C:0A:F3:3C:C1:A7
IP	10.0.0.103
Hostname	N/A

Field	Value
OS fingerprint	N/A

Open ports

None recorded.

Watchdog mini-recap

- **Exposed:** 0 open port(s), 0 high-risk service(s).
- **Vulnerable:** 0 finding(s) (Critical 0, High 0, Medium 0).
- **Evidence:** 0/0 finding(s) include evidence snippets.
- **Remediation:** 0/0 finding(s) include remediation guidance.

Vulnerabilities

CVE/CVSS are shown when available; CVSS may be estimated from severity when not provided by the source.

None found.

Asset: VAPTOR001A0004

Asset fingerprint

Field	Value
MAC Address	16:41:0F:EC:FD:25
IP	10.0.0.113
Hostname	N/A
OS fingerprint	N/A

Open ports

None recorded.

Watchdog mini-recap

- **Exposed:** 0 open port(s), 0 high-risk service(s).
- **Vulnerable:** 0 finding(s) (Critical 0, High 0, Medium 0).
- **Evidence:** 0/0 finding(s) include evidence snippets.
- **Remediation:** 0/0 finding(s) include remediation guidance.

Vulnerabilities

CVE/CVSS are shown when available; CVSS may be estimated from severity when not provided by the source.

None found.

Asset: VAPTOR001A0005

Asset fingerprint

Field	Value
MAC Address	42:27:0A:23:DF:92
IP	10.0.0.117
Hostname	N/A
OS fingerprint	N/A

Open ports

None recorded.

Watchdog mini-recap

- **Exposed:** 0 open port(s), 0 high-risk service(s).
- **Vulnerable:** 0 finding(s) (Critical 0, High 0, Medium 0).
- **Evidence:** 0/0 finding(s) include evidence snippets.

- **Remediation:** 0/0 finding(s) include remediation guidance.

Vulnerabilities

CVE/CVSS are shown when available; CVSS may be estimated from severity when not provided by the source.

None found.

Asset: VAPTOR001A0006

Asset fingerprint

Field	Value
MAC Address	2A:DD:35:08:D3:72
IP	10.0.0.120
Hostname	N/A
OS fingerprint	Android 12 (Linux 5.4)

Open ports

Proto	Port	Service	Risk
tcp	6700	carracho	—

Watchdog mini-recap

- **Exposed:** 1 open port(s), 0 high-risk service(s).
  Top: tcp/6700 carracho
- **Vulnerable:** 0 finding(s) (Critical 0, High 0, Medium 0).
- **Evidence:** 0/0 finding(s) include evidence snippets.
- **Remediation:** 0/0 finding(s) include remediation guidance.

Vulnerabilities

CVE/CVSS are shown when available; CVSS may be estimated from severity when not provided by the source.

None found.

Asset: VAPTOR001A0007

Asset fingerprint

Field	Value
MAC Address	70:F8:2B:1B:A3:68

Field	Value
IP	10.0.0.254
Hostname	www.adsl.vf
OS fingerprint	Linux 2.6.32 - 3.13

Open ports

Proto	Port	Service	Risk
tcp	53	domain	MEDIUM
tcp	80	http	HIGH
tcp	443	https	MEDIUM
tcp	1990	stun-p1	—

High-risk port notes (VaptorAI / Intel DB)

Port exposure notes (VaptorAI): tcp/80 http [HIGH]

Narrative

```
# Port Exposure: tcp/80
```

```
## Summary
An Internet-accessible service was detected on tcp/80. Exposed management and remote-
access ports are frequently targeted for credential attacks and exploitation.

## Why this matters (Risk)
- Increases attack surface and brute-force exposure
- May permit remote code execution if vulnerable and unpatched
- Often enables lateral movement once credentials are obtained

## Recommended actions (Remediation)
- Restrict access (VPN/jump host/allowlist)
- Enforce MFA and strong authentication policies
- Patch the exposed service and disable legacy/insecure options

## Evidence
- Observed open: tcp/80
```

Port exposure notes: tcp/80 http [HIGH]

Why this matters Unencrypted web traffic; web application vulnerabilities, MitM attacks

Common attacksSQL Injection, XSS, CSRF, Directory Traversal, Man-in-the-Middle, Session Hijacking

Notable CVEsCVE-2021-44228, CVE-2017-5638, CVE-2014-0160

Recommended action Redirect all traffic to HTTPS (port 443). Disable HTTP or use only for redirect. Implement HSTS headers.

Watchdog mini-recap

- **Exposed:** 4 open port(s), 1 high-risk service(s).
  Top: tcp/53 domain [MEDIUM]; tcp/80 http [HIGH]; tcp/443 https [MEDIUM]
- **Vulnerable:** 0 finding(s) (Critical 0, High 0, Medium 0).
- **Evidence:** 0/0 finding(s) include evidence snippets.
- **Remediation:** 0/0 finding(s) include remediation guidance.

Vulnerabilities

CVE/CVSS are shown when available; CVSS may be estimated from severity when not provided by the source.

None found.

Asset: VAPTOR001A0008

Asset fingerprint

Field	Value
MAC Address	B8:27:EB:74:F2:6C

Field	Value
IP	10.0.0.132
Hostname	N/A
OS fingerprint	Linux 4.15 - 5.19

Open ports

Proto	Port	Service	Risk
tcp	22	ssh	HIGH

High-risk port notes (VaptorAI / Intel DB)

Port exposure notes (VaptorAI): tcp/22 ssh [HIGH]

Narrative

```
# Port Exposure: tcp/22

## Summary
An Internet-accessible service was detected on tcp/22. Exposed management and remote-
access ports are frequently targeted for credential attacks and exploitation.

## Why this matters (Risk)
```

```
- Increases attack surface and brute-force exposure
- May permit remote code execution if vulnerable and unpatched
- Often enables lateral movement once credentials are obtained

## Recommended actions (Remediation)
- Restrict access (VPN/jump host/allowlist)
- Enforce MFA and strong authentication policies
- Patch the exposed service and disable legacy/insecure options

## Evidence
- Observed open: tcp/22
```

Port exposure notes: tcp/22 ssh [HIGH]

Why this matters Remote shell access; brute-force, key reuse, weak credentials

Common attacks Brute Force, Credential Stuffing, Private Key Theft, Man-in-the-Middle

Notable CVEsCVE-2016-20012, CVE-2018-15473, CVE-2021-28041

Recommended action Implement key-based authentication only. Disable password authentication. Use fail2ban. Restrict access by IP. Enable 2FA.

Watchdog mini-recap

- **Exposed:** 1 open port(s), 1 high-risk service(s).
  Top: tcp/22 ssh [HIGH]
- **Vulnerable:** 0 finding(s) (Critical 0, High 0, Medium 0).
- **Evidence:** 0/0 finding(s) include evidence snippets.
- **Remediation:** 0/0 finding(s) include remediation guidance.

Vulnerabilities

CVE/CVSS are shown when available; CVSS may be estimated from severity when not provided by the source.

None found.

Asset: VAPTOR001A0009

Asset fingerprint

Field	Value
MAC Address	F4:63:1F:59:71:1C
IP	10.0.0.106
Hostname	N/A

Field	Value
OS fingerprint	Citrix Access Gateway VPN gateway

Open ports

None recorded.

Watchdog mini-recap

- **Exposed:** 0 open port(s), 0 high-risk service(s).
- **Vulnerable:** 0 finding(s) (Critical 0, High 0, Medium 0).
- **Evidence:** 0/0 finding(s) include evidence snippets.
- **Remediation:** 0/0 finding(s) include remediation guidance.

Vulnerabilities

CVE/CVSS are shown when available; CVSS may be estimated from severity when not provided by the source.

None found.

Asset: VAPTOR001A0010

Asset fingerprint

Field	Value
MAC Address	38:53:9C:B8:D1:98
IP	10.0.0.108
Hostname	N/A
OS fingerprint	Apple macOS 11 (Big Sur) - 13 (Ventura) or iOS 16 (Darwin 20.6.0 - 22.4.0)

Open ports

Proto	Port	Service	Risk
tcp	49152	—	—
tcp	62078	iphone-sync	—

Watchdog mini-recap

- **Exposed:** 2 open port(s), 0 high-risk service(s).
  Top: tcp/49152 —; tcp/62078 iphone-sync
- **Vulnerable:** 0 finding(s) (Critical 0, High 0, Medium 0).
- **Evidence:** 0/0 finding(s) include evidence snippets.

- **Remediation:** 0/0 finding(s) include remediation guidance.

Vulnerabilities

CVE/CVSS are shown when available; CVSS may be estimated from severity when not provided by the source.

None found.

3b) Audit (AI) - Log Analysis

AI-assisted analysis of uploaded security logs for this scan. Reports list audit artifacts only (not raw log contents) to reduce sensitive-data exposure.

Analysis Summary

Analysis completed: 2026-03-20 11:41:52 UTC

Files processed: 210

Vulnerabilities found: 3

Threats detected: 5

Recommendations: 4

Analyzed 25 scan-folder artifact files. Found 21 vulnerabilities, 29 threats, and 39 recommendations.

Security Vulnerabilities Identified (3)

Title	Severity	Description	Source Artifact
CVE reference found	HIGH	CVE reference found	wapiti_results.json
Weak configuration/credential	MEDIUM	Weak configuration/credential	wapiti_results.json
Vulnerability mention	MEDIUM	Vulnerability mention	wapiti_results.json

Security Threats Detected (5)

Threat	Severity	Category	Source Artifact
Access denied events found	MEDIUM	denied	pcap_summary.json, VAPTORPCAP20260320113522.log
Exploit attempt signatures	MEDIUM	exploit	scanner_execution.json, wapiti_results.json
Unauthorized access attempts	MEDIUM	unauthorized	wapiti_results.json
Potential attack indicators	MEDIUM	attack	wapiti_results.json
Injection attempt patterns	MEDIUM	injection	wapiti_results.json

Security Recommendations (4)

Priority	Category	Recommended Action	Source File
HIGH	Security	Investigate threat indicators immediately	—
HIGH	Patching	Apply patches and security updates for identified vulnerabilities	—
MEDIUM	Monitoring	Review anomalies for potential system or security issues	—
LOW	Monitoring	Continue regular monitoring and log review	—

4) Compliance Evaluation & Score

This section is informational and helps triage findings against control frameworks. It is not a formal compliance audit. Controls are marked **Non-Compliant** when mapped findings exist for this scan; controls only become **Compliant** when audit evidence (vaptaudit) explicitly marks them as PASS.

Metric	Value
Percent compliant	4% (Low)
Control coverage	5/74 (7%)
Vulnerability-to-control mapping	0/39 (0%)

Metric	Value
Auto mapping (bootstrap)	No

Compliance Summary by Framework

Framework	Compliant	Non-Compliant	Unmapped
HIPAA	0/57 (0%)	4	53
ISO-27001	0/5 (0%)	0	5
NIST-CSF	0/7 (0%)	1	6
PCI-DSS	0/5 (0%)	0	5

Compliance Controls (Addendum)

Status legend: **Compliant**=Informational: vaptaudit provided PASS evidence for this control for this scan.; **Non-Compliant**=Informational: a mapped finding impacts this control for this scan (or vaptaudit marked FAIL).; **Unmapped**=Informational: no mapped findings for this control and no vaptaudit PASS evidence.

Framework	Control Title	Affected AssetUID	Status
HIPAA	Security Management Process	VAPTOR001A0002	Non-Compliant

Framework	Control Title	Affected AssetUID	Status
HIPAA	Security Management Process - Risk Analysis	VAPTOR001A0002	Non-Compliant
HIPAA	Security Management Process - Risk Management	—	Unmapped
HIPAA	Security Management Process - Sanction Policy	—	Unmapped
HIPAA	Security Management Process - Information System Activity Review	—	Unmapped
HIPAA	Assigned Security Responsibility	—	Unmapped
HIPAA	Workforce Security	—	Unmapped
HIPAA	Workforce Security - Authorization and Supervision	—	Unmapped
HIPAA	Workforce Security - Clearance Procedure	—	Unmapped
HIPAA	Workforce Security - Termination Procedures	—	Unmapped
HIPAA	Information Access Management	—	Unmapped
HIPAA	Information Access Management - Access Authorization	—	Unmapped
HIPAA		—	Unmapped

Framework	Control Title	Affected AssetUID	Status
	Information Access Management - Access Establishment and Modification		
HIPAA	Security Awareness and Training	—	Unmapped
HIPAA	Security Awareness and Training	—	Unmapped
HIPAA	Security Awareness and Training - Security Reminders	—	Unmapped
HIPAA	Security Awareness and Training - Protection from Malicious Software	—	Unmapped
HIPAA	Security Awareness and Training - Log-in Monitoring	—	Unmapped
HIPAA	Security Awareness and Training - Password Management	—	Unmapped
HIPAA	Security Incident Procedures	—	Unmapped
HIPAA	Security Incident Procedures - Response and Reporting	VAPTOR001A0002	Non-Compliant
HIPAA	Contingency Plan	—	Unmapped
HIPAA	Contingency Plan - Data Backup	—	Unmapped
HIPAA	Contingency Plan - Disaster Recovery	—	Unmapped

Framework	Control Title	Affected AssetUID	Status
HIPAA	Contingency Plan - Emergency Mode Operation	—	Unmapped
HIPAA	Contingency Plan - Testing and Revision	—	Unmapped
HIPAA	Evaluation	—	Unmapped
HIPAA	Business Associate Contracts	—	Unmapped
HIPAA	Facility Access Controls	—	Unmapped
HIPAA	Facility Access Controls - Contingency Operations	—	Unmapped
HIPAA	Facility Access Controls - Facility Security Plan	—	Unmapped
HIPAA	Facility Access Controls - Access Control and Validation	—	Unmapped
HIPAA	Facility Access Controls - Maintenance Records	—	Unmapped
HIPAA	Workstation Use	VAPTOR001A0002	Non-Compliant
HIPAA	Workstation Security	—	Unmapped
HIPAA	Device and Media Controls - Disposal	—	Unmapped
HIPAA	Device and Media Controls - Media Re-use	—	Unmapped

Framework	Control Title	Affected AssetUID	Status
HIPAA	Device and Media Controls - Accountability	—	Unmapped
HIPAA	Device and Media Controls - Data Backup and Storage	—	Unmapped
HIPAA	Access Control	—	Unmapped
HIPAA	Access Control - Unique User Identification	—	Unmapped
HIPAA	Access Control - Emergency Access Procedure	—	Unmapped
HIPAA	Access Control - Automatic Logoff	—	Unmapped
HIPAA	Access Control - Encryption and Decryption	—	Unmapped
HIPAA	Audit Controls	—	Unmapped
HIPAA	Integrity	—	Unmapped
HIPAA	Integrity - Mechanism to Authenticate ePHI	—	Unmapped
HIPAA	Person or Entity Authentication	—	Unmapped
HIPAA	Transmission Security	—	Unmapped
HIPAA	Transmission Security - Integrity Controls	—	Unmapped

Framework	Control Title	Affected AssetUID	Status
HIPAA	Transmission Security - Encryption	—	Unmapped
HIPAA	Business Associate Contracts - Written Contract	—	Unmapped
HIPAA	Requirements for Group Health Plans	—	Unmapped
HIPAA	Policies and Procedures	—	Unmapped
HIPAA	Documentation - Time Limit	—	Unmapped
HIPAA	Documentation - Availability	—	Unmapped
HIPAA	Documentation - Updates	—	Unmapped
ISO-27001	Policies for information security	—	Unmapped
ISO-27001	Screening	—	Unmapped
ISO-27001	Physical security perimeters	—	Unmapped
ISO-27001	User endpoint devices	—	Unmapped
ISO-27001	Privileged access rights	—	Unmapped
NIST-CSF	Networks are monitored	—	Unmapped

Framework	Control Title	Affected AssetUID	Status
NIST-CSF	Asset inventories are maintained	—	Unmapped
NIST-CSF	Software platforms are inventoried	VAPTOR001A0002	Non-Compliant
NIST-CSF	Identities are managed	—	Unmapped
NIST-CSF	Remote access is managed	—	Unmapped
NIST-CSF	Recovery plan is executed	—	Unmapped
NIST-CSF	Response plan is executed	—	Unmapped
PCI-DSS	Firewall configuration standards	—	Unmapped
PCI-DSS	Change default passwords	—	Unmapped
PCI-DSS	CHD retention policy	—	Unmapped
PCI-DSS	Strong cryptography for CHD transmission	—	Unmapped
PCI-DSS	Secure development processes	—	Unmapped

Vulnerability → Control Trace (This Scan)

This links findings to impacted controls so users can see why a control is flagged as Non-Compliant.

AssetUID	Severity	Finding	Impacted Controls
VAPTOR001A0002	INFORMATIONAL	User Controllable HTML Element Attribute (Potential XSS)	HIPAA: Workstation Use
VAPTOR001A0002	INFORMATIONAL	User Agent Fuzzer	HIPAA: Security Management Process
VAPTOR001A0002	INFORMATIONAL	Session Management Response Identified	HIPAA: Security Management Process
VAPTOR001A0002	INFORMATIONAL	GET for POST	HIPAA: Security Management Process
VAPTOR001A0002	INFORMATIONAL	Charset Mismatch (Header Versus Meta Content-Type Charset)	HIPAA: Security Management Process
VAPTOR001A0002	INFORMATIONAL	Authentication Request Identified	HIPAA: Security Management Process
VAPTOR001A0002	LOW	X-Content-Type-Options Header Missing	HIPAA: Security Management Process
VAPTOR001A0002	LOW	X-AspNet-Version Response Header	HIPAA: Security Incident Procedures - Response and Reporting

AssetUID	Severity	Finding	Impacted Controls
VAPTOR001A0002	LOW	Server Leaks Version Information via "Server" HTTP Response Header Field	HIPAA: Security Management Process - Risk Analysis
VAPTOR001A0002	LOW	Server Leaks Information via "X-Powered-By" HTTP Response Header Field(s)	HIPAA: Security Management Process - Risk Analysis
VAPTOR001A0002	LOW	Cookie without SameSite Attribute	HIPAA: Workstation Use
VAPTOR001A0002	MEDIUM	Missing Anti-clickjacking Header	HIPAA: Security Management Process
VAPTOR001A0002	MEDIUM	HTTP Only Site	HIPAA: Security Management Process
VAPTOR001A0002	MEDIUM	Content Security Policy (CSP) Header Not Set	HIPAA: Security Management Process
VAPTOR001A0002	MEDIUM	Absence of Anti-CSRF Tokens	HIPAA: Security Management Process
VAPTOR001A0002	HIGH	SQL Injection - SQLite (Time Based)	HIPAA: Security Management Process
VAPTOR001A0002	HIGH	SQL Injection - MsSQL (Time Based)	

AssetUID	Severity	Finding	Impacted Controls
			HIPAA: Security Management Process
VAPTOR001A0002	HIGH	SQL Injection	HIPAA: Security Management Process
VAPTOR001A0002	HIGH	Cross Site Scripting (Reflected)	HIPAA: Security Management Process
VAPTOR001A0002	HIGH	Cross Site Scripting (Persistent)	HIPAA: Security Management Process
VAPTOR001A0002	INFORMATIONAL	User Agent Fuzzer	HIPAA: Security Management Process
VAPTOR001A0002	INFORMATIONAL	Session Management Response Identified	HIPAA: Security Management Process
VAPTOR001A0002	INFORMATIONAL	Information Disclosure - Suspicious Comments	HIPAA: Security Management Process - Risk Analysis
VAPTOR001A0002	LOW	X-Content-Type-Options Header Missing	HIPAA: Security Management Process
VAPTOR001A0002	LOW	Server Leaks Version Information via "Server" HTTP Response Header Field	HIPAA: Security Management Process - Risk Analysis

AssetUID	Severity	Finding	Impacted Controls
VAPTOR001A0002	LOW	Server Leaks Information via "X-Powered-By" HTTP Response Header Field(s)	HIPAA: Security Management Process - Risk Analysis
VAPTOR001A0002	LOW	Information Disclosure - Debug Error Messages	HIPAA: Security Management Process - Risk Analysis
VAPTOR001A0002	LOW	Cookie without SameSite Attribute	HIPAA: Workstation Use
VAPTOR001A0002	LOW	Cookie No HttpOnly Flag	HIPAA: Security Management Process
VAPTOR001A0002	LOW	Application Error Disclosure	NIST-CSF: Software platforms are inventoried
VAPTOR001A0002	MEDIUM	Missing Anti-clickjacking Header	HIPAA: Security Management Process
VAPTOR001A0002	MEDIUM	HTTP Only Site	HIPAA: Security Management Process
VAPTOR001A0002	MEDIUM	Content Security Policy (CSP) Header Not Set	HIPAA: Security Management Process
VAPTOR001A0002	MEDIUM	Absence of Anti-CSRF Tokens	

AssetUID	Severity	Finding	Impacted Controls
			HIPAA: Security Management Process
VAPTOR001A0002	HIGH	SQL Injection - MsSQL (Time Based)	HIPAA: Security Management Process
VAPTOR001A0002	HIGH	SQL Injection	HIPAA: Security Management Process
VAPTOR001A0002	HIGH	Path Traversal	HIPAA: Security Management Process
VAPTOR001A0002	HIGH	External Redirect	HIPAA: Security Management Process
VAPTOR001A0002	HIGH	Cross Site Scripting (Reflected)	HIPAA: Security Management Process

5) Remediation & Compliance Roadmap

1. Restrict exposure of 10.0.0.132:22 (HIGH). Implement key-based authentication only. Disable password authentication. Use fail2ban. Restrict access by IP. Enable 2FA.
2. Restrict exposure of 10.0.0.254:80 (HIGH). Redirect all traffic to HTTPS (port 443). Disable HTTP or use only for redirect. Implement HSTS headers.

3. Restrict exposure of 44.238.29.244:80 (HIGH). Redirect all traffic to HTTPS (port 443). Disable HTTP or use only for redirect. Implement HSTS headers.
4. Patch or mitigate CRITICAL/HIGH vulnerabilities first; validate fixes with a re-scan.
5. Harden authentication: enforce MFA where possible, disable default credentials, and add account lockout policies for remote services.
6. Compliance: ensure control mappings are configured (set `compliance.default_control_id`) and remediate items mapped to highest-priority controls first.

Appendix A — Scan Artifact Index

Complete inventory of scan artifacts generated during this assessment.

Artifact Path	Size (bytes)
artifacts/ai_ready_summary.json	25,143
artifacts/api_scan.json	386
artifacts/app/apkleaks_results.json	0
artifacts/app/appsec_summary.json	3,915
artifacts/app/static_analysis.json	2,939
artifacts/VAPTORDISCOVERY.txt	440

Artifact Path	Size (bytes)
artifacts/consolidated_vulnerabilities.json	149,636
artifacts/consolidated_vulnerabilities_penetration_test.json	1,367
artifacts/consolidated_vulnerabilities_vulnerability_assessment.json	149,622
artifacts/dast/dast_summary.json	654
artifacts/dast/nuclei_results.json	0
artifacts/dast/nuclei_targets.txt	180
artifacts/dast/wapiti_1_http_testhtml5.vulnweb.com/wapiti_results.json	42,217
artifacts/dast/wapiti_2_http_testphp.vulnweb.com/wapiti_results.json	42,215
artifacts/dast/wapiti_3_http_testasp.vulnweb.com/wapiti_results.json	42,215
artifacts/dast/wapiti_4_http_testaspnet.vulnweb.com/wapiti_results.json	42,218
artifacts/dast/wapiti_5_http_10.0.0.254_80/wapiti_results.json	42,206
artifacts/dast/wapiti_6_https_10.0.0.254_443/wapiti_results.json	42,207
artifacts/dast/wapiti_7_http_44.238.29.244_80/wapiti_results.json	42,209

Artifact Path	Size (bytes)
artifacts/dast/wapiti_summary.json	1,921
artifacts/discovery/_nmap_chunks/VAPTORDISCOVERY-part-1.xml	141,626
artifacts/discovery/_nmap_chunks/VAPTORDISCOVERY-part-2.xml	45,164
artifacts/discovery/discovery.csv	531
artifacts/discovery/discovery.json	25,039
artifacts/discovery/discovery.yaml	925
artifacts/discovery/httpx_targets.txt	181
artifacts/discovery/httpx_web.jsonl	1,873
artifacts/discovery/VAPTORDISCOVERY.xml	169,421
artifacts/discovery/websites.json	1,933
artifacts/discovery.csv	531
artifacts/discovery.json	25,039
artifacts/enrichment/1990.json	1,901

Artifact Path	Size (bytes)
artifacts/enrichment/22.json	1,874
artifacts/enrichment/443.json	1,892
artifacts/enrichment/49152.json	1,882
artifacts/enrichment/53.json	1,886
artifacts/enrichment/62078.json	1,903
artifacts/enrichment/6700.json	1,893
artifacts/enrichment/80.json	1,996
artifacts/enrichment/builtwith.json	806
artifacts/enrichment/vuln_0a7c8d77a2ad.json	2,123
artifacts/enrichment/vuln_1c20db268bb6.json	2,189
artifacts/enrichment/vuln_1fa250c2cdaf.json	2,215
artifacts/enrichment/vuln_27d6e876416c.json	2,277
artifacts/enrichment/vuln_2b9ba30fd468.json	2,258

Artifact Path	Size (bytes)
artifacts/enrichment/vuln_2c5100794604.json	2,154
artifacts/enrichment/vuln_2d9220dcd958.json	2,189
artifacts/enrichment/vuln_2ffdea09c85b.json	2,124
artifacts/enrichment/vuln_31642d28952d.json	2,226
artifacts/enrichment/vuln_33d6c45ccda9.json	2,114
artifacts/enrichment/vuln_3e9181c589a1.json	2,158
artifacts/enrichment/vuln_3ebdb84b8df2.json	2,150
artifacts/enrichment/vuln_41122ade5880.json	2,198
artifacts/enrichment/vuln_4223f535ee2b.json	2,144
artifacts/enrichment/vuln_47bcd63bc4cb.json	2,141
artifacts/enrichment/vuln_489aa29b9d65.json	2,191
artifacts/enrichment/vuln_549aedbb8c7d.json	2,254
artifacts/enrichment/vuln_5805db5e0f3b.json	2,130

Artifact Path	Size (bytes)
artifacts/enrichment/vuln_5d12c102cbe5.json	2,304
artifacts/enrichment/vuln_6aab9e532f4d.json	2,205
artifacts/enrichment/vuln_797f39e3a2a8.json	2,259
artifacts/enrichment/vuln_7b0599c6dafd.json	2,256
artifacts/enrichment/vuln_87d369c2db70.json	2,132
artifacts/enrichment/vuln_8f025c43e29a.json	2,166
artifacts/enrichment/vuln_9134065e0f87.json	2,143
artifacts/enrichment/vuln_9536bb0af811.json	2,209
artifacts/enrichment/vuln_a9606ca282c8.json	2,238
artifacts/enrichment/vuln_b2dd3660ea63.json	2,195
artifacts/enrichment/vuln_b3ff91233170.json	2,247
artifacts/enrichment/vuln_b51f0107404a.json	2,271
artifacts/enrichment/vuln_bb575fa50bd5.json	2,355

Artifact Path	Size (bytes)
artifacts/enrichment/vuln_c59cffeff900.json	2,202
artifacts/enrichment/vuln_cc2f12f5d425.json	2,156
artifacts/enrichment/vuln_d25b833c8ea7.json	2,175
artifacts/enrichment/vuln_d8df761fc465.json	2,145
artifacts/enrichment/vuln_e160cb1a2d5b.json	2,133
artifacts/enrichment/vuln_ec2e554fed03.json	2,305
artifacts/enrichment/vuln_fe80fd3fe58c.json	2,250
artifacts/enrichment/vuln_fedc956996ce.json	2,167
artifacts/enrichment/whois.json	3,428
artifacts/enrichment.json	2,346
artifacts/VAPTORMSF/auto_10_0_0_108.rc	150
artifacts/VAPTORMSF/auto_10_0_0_120.rc	143
artifacts/VAPTORMSF/auto_10_0_0_132.rc	225

Artifact Path	Size (bytes)
artifacts/VAPTORMSF/auto_10_0_0_254.rc	475
artifacts/VAPTORMSF/auto_44_238_29_244.rc	320
artifacts/VAPTORMSF/msf_10_0_0_108.txt	1,545
artifacts/VAPTORMSF/msf_10_0_0_120.txt	1,419
artifacts/VAPTORMSF/msf_10_0_0_132.txt	2,298
artifacts/VAPTORMSF/msf_10_0_0_254.txt	4,783
artifacts/VAPTORMSF/msf_44_238_29_244.txt	3,220
artifacts/metasploit_auth/auth_10_0_0_132.rc	324
artifacts/metasploit_auth/msf_auth_10_0_0_132.txt	2,191
artifacts/metasploit_auth/userpass_trimmed.userpass.txt	7,068
artifacts/mitm/VAPTORARP20260320113456.log	45,395,105
artifacts/mitm/VAPTORARP20260320113522.log	728
artifacts/mitm/VAPTORCAP20260320113522.pcapng	0

Artifact Path	Size (bytes)
artifacts/mitm/VAPTORPCAP20260320113522.log	532
artifacts/mitm/pcap_summary.json	723
artifacts/mitm/recon_observations.json	13,658
artifacts/msf_auth.json	2,024
artifacts/msf_verification.json	3,026
artifacts/VAPTORDISCOVERY.txt	91
artifacts/phases_summary.json	1,868
artifacts/sast/.gitleaks.toml	541
artifacts/sast/bandit_results.json	407
artifacts/sast/bearer_results.json	1,677
artifacts/sast/gitleaks_results.json	3
artifacts/sast/sast_summary.json	776
artifacts/scan_events.jsonl	1,664

Artifact Path	Size (bytes)
artifacts/scanner_execution.json	3,229
artifacts/scanner_validation.json	571
artifacts/scanner_validation_penetration_test.json	541
artifacts/scanner_validation_vulnerability_assessment.json	259
artifacts/sqlmap_http___testasp.vulnweb.com/testasp.vulnweb.com/log	0
artifacts/sqlmap_http___testasp.vulnweb.com/testasp.vulnweb.com/session.sqlite	8,192
artifacts/sqlmap_http___testasp.vulnweb.com/testasp.vulnweb.com/target.txt	249
artifacts/sqlmap_http___testaspnet.vulnweb.com/testaspnet.vulnweb.com/log	0
artifacts/sqlmap_http___testaspnet.vulnweb.com/testaspnet.vulnweb.com/session.sqlite	8,192
artifacts/sqlmap_http___testaspnet.vulnweb.com/testaspnet.vulnweb.com/target.txt	258
artifacts/sqlmap_results.json	339
artifacts/vaptorgpt.json	38,139
artifacts/vaptorset/campaign_results.json	1,653

Artifact Path	Size (bytes)
artifacts/vaptorset/set_summary.json	2,683
artifacts/vaptorset/templates/Xneelo.html	7,665
artifacts/vaptorset/templates/facebook.html	1,352
artifacts/vaptorset/templates/gmail.html	1,671
artifacts/vaptorset/templates/https___webmail_konsoleh_co_za_login.html	7,665
artifacts/vaptorset/templates/office365.html	1,447
artifacts/vaptorset/templates/twitter.html	1,264
artifacts/vaptorset/web_clone/Xneelo/backblue.gif	4,243
artifacts/vaptorset/web_clone/Xneelo/fade.gif	828
artifacts/vaptorset/web_clone/Xneelo/hts-cache/doit.log	475
artifacts/vaptorset/web_clone/Xneelo/hts-cache/new.lst	112
artifacts/vaptorset/web_clone/Xneelo/hts-cache/new.txt	891
artifacts/vaptorset/web_clone/Xneelo/hts-cache/new.zip	3,055

Artifact Path	Size (bytes)
artifacts/vaptorset/web_clone/Xneelo/hts-log.txt	880
artifacts/vaptorset/web_clone/Xneelo/index.html	5,227
artifacts/vaptorset/web_clone/Xneelo/webmail.konsoleh.co.za/favicon.ico	1,150
artifacts/vaptorset/web_clone/Xneelo/webmail.konsoleh.co.za/login.html	7,653
artifacts/vaptorset/web_clone/https___webmail_konsoleh_co_za_login/backblue.gif	4,243
artifacts/vaptorset/web_clone/https___webmail_konsoleh_co_za_login/fade.gif	828
artifacts/vaptorset/web_clone/https___webmail_konsoleh_co_za_login/hts-cache/doit.log	505
artifacts/vaptorset/web_clone/https___webmail_konsoleh_co_za_login/hts-cache/new.lst	112
artifacts/vaptorset/web_clone/https___webmail_konsoleh_co_za_login/hts-cache/new.txt	951
artifacts/vaptorset/web_clone/https___webmail_konsoleh_co_za_login/hts-cache/new.zip	3,055
artifacts/vaptorset/web_clone/https___webmail_konsoleh_co_za_login/hts-log.txt	910
artifacts/vaptorset/web_clone/https___webmail_konsoleh_co_za_login/index.html	5,227
artifacts/vaptorset/web_clone/https___webmail_konsoleh_co_za_login/webmail.konsoleh.co.za/favicon.ico	1,150

Artifact Path	Size (bytes)
artifacts/vaptorset/web_clone/https___webmail_konsoleh_co_za_login/webmail.konsoleh.co.za/login.html	7,653
artifacts/waf_vhost.json	2,778
artifacts/web_recon.json	4,042
artifacts/web_target_gating.json	904
artifacts/websites.json	1,933
artifacts/zap_http_testasp.vulnweb.com.json	67,227
artifacts/zap_http_testaspnet.vulnweb.com.json	120,040
artifacts/zap_results.json	73,748
audit/audit_analysis.json	435,685
audit/include_in_report.flag	8